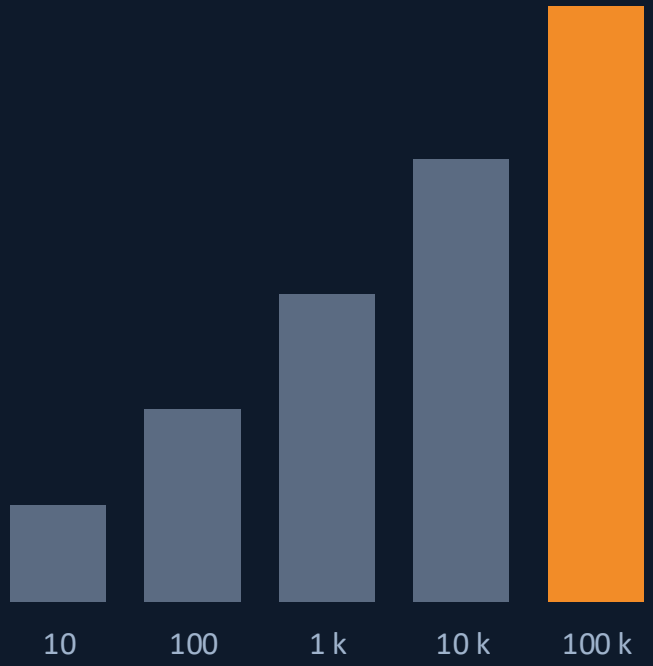
De 10 à 100 000 utilisateurs

Qu'est-ce qui poserait problème, et comment l'éviter ?



Ce qui casse entre 10 et 100 000

×10 000

plus d'utilisateurs, donc des milliers de fois plus de requêtes et de données

Ajouter du CPU ne suffit pas : l'architecture doit changer.



Backend

un seul serveur sature



Base de données

requêtes lentes, connexions épuisées



Stockage

disque plein, fichiers non partagés



Sessions

perdues d'une instance à l'autre



Cache

absent : la base encaisse tout



Déploiement

manuel, avec coupure de service

Backend : fin du serveur unique



Point de défaillance unique

CPU et RAM saturés ; une panne et tout est hors ligne.



Traitements lourds dans la requête

Envoi d'e-mails, PDF, images : les temps de réponse explosent.



État gardé en mémoire

Impossible d'ajouter des instances sans casser l'application.

Ce qu'il faut faire

- Backend stateless
- Plusieurs instances derrière un load balancer
- Tâches lourdes en asynchrone (file + workers)
- Timeouts, rate limiting, pagination

Base de données : le premier goulot



Requêtes lentes

PROBLÈME

Full scans et requêtes N+1 : le temps de réponse explose.

SOLUTION

Index ciblés, EXPLAIN, pagination, requêtes groupées.



Connexions saturées

PROBLÈME

Chaque instance ouvre ses connexions : la limite du serveur est atteinte.

SOLUTION

Pool de connexions (PgBouncer) et limites par instance.



Un seul nœud

PROBLÈME

Lectures et écritures sur la même machine : plus aucune marge.

SOLUTION

Réplicas en lecture, puis partitionnement si nécessaire.

Règle d'or : mesurer, optimiser, puis seulement scaler.

Stockage : sortir du disque local

À 10 utilisateurs

- Uploads sur le disque du serveur
- Disque plein = application en panne
- Fichiers perdus si le serveur est recréé
- Sauvegardes manuelles



À 100 000 utilisateurs

- Object storage (S3, MinIO)
- CDN devant les médias et le statique
- URLs signées pour les accès privés
- BDD = métadonnées uniquement
- Sauvegardes et cycle de vie automatisés

Un disque local ne se partage pas entre plusieurs instances.

Sessions et cache : partager l'état



Sessions

PROBLÈME

Sessions en mémoire ou en fichiers locaux : l'utilisateur est déconnecté en changeant d'instance ou à chaque déploiement.

SOLUTION

- Store central Redis partagé
- Ou JWT stateless : jeton court + refresh
- Fin des sticky sessions



Cache

PROBLÈME

Les mêmes données sont relues des milliers de fois : la base encaisse tout le trafic.

SOLUTION

- Cache-aside Redis avec TTL
- Cache HTTP et CDN pour le statique
- Prévoir l'invalidation du cache

Pas de scalabilité horizontale tant que l'état reste dans les instances.

Déploiement : automatiser et sécuriser



À 10 utilisateurs : déploiement manuel (SSH, git pull), service coupé, aucun retour arrière possible.

1. Build

Image Docker reproductible, même artefact partout

2. Test

Pipeline CI/CD : tests automatiques à chaque commit

3. Release

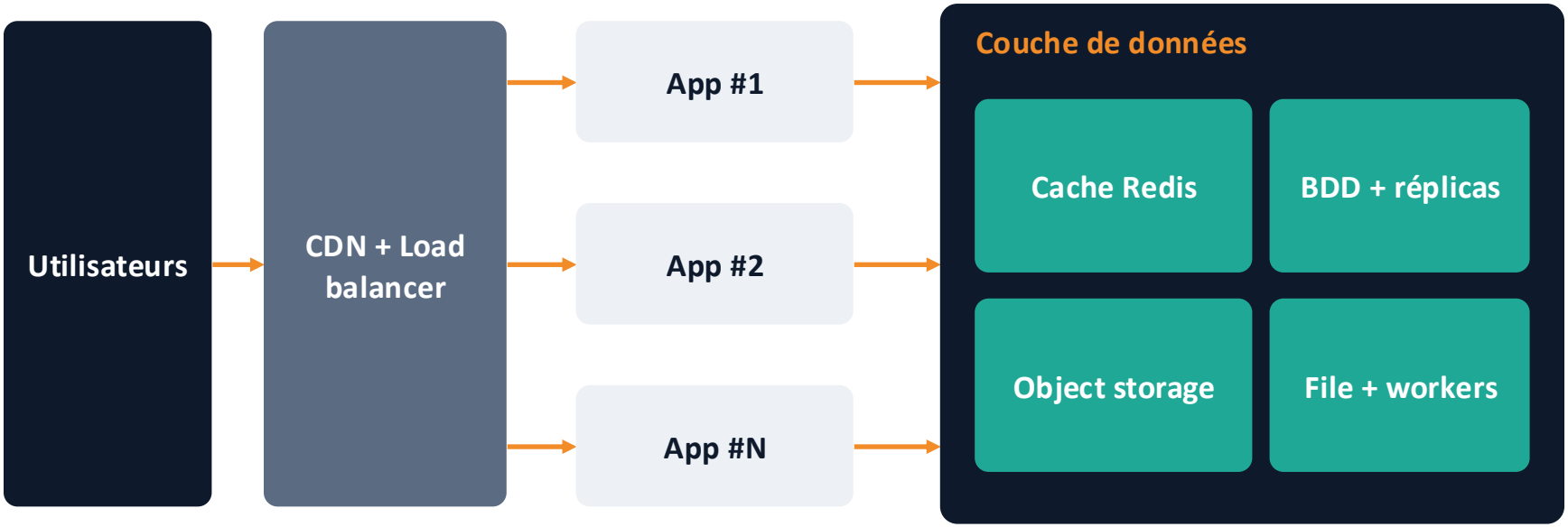
Rolling, blue-green ou canary : jamais tout d'un coup

4. Observer

Logs, métriques, alertes ; rollback en un clic

Autoscaling et conteneurs (Docker, Kubernetes) : la capacité suit la charge.

Pistes de scalabilité



Vertical : un plus gros serveur, vite limité

Horizontal : plus d'instances, c'est la cible

Scaler par étapes, pas d'un coup

MAINTENANT

10 → 1 000

- Backend stateless
- Index et requêtes saines
- CI/CD et sauvegardes

ENSUITE

1 000 → 10 000

- Redis : cache et sessions
- Object storage + CDN
- Load balancer + réplicas

À 100 000

et au-delà

- Autoscaling
- Files et workers
- Monitoring, sharding si besoin

Mesurer d'abord, puis scaler ce qui sature réellement.